Notatka do zadania 2 — system zarządzania danymi przestrzennymi

O co chodzi w tym projekcie

Ten projekt to prosty system bazy danych do zarządzania **parkami**, **ścieżkami w parkach** i **obiektami znajdującymi się w parkach**.

Obiektami mogą być na przykład: ławki, fontanny, place zabaw, pomosty albo stacje rowerowe.

Najważniejsza cecha tego zadania jest taka, że baza nie przechowuje tylko zwykłych napisów i liczb, ale także **dane przestrzenne**, czyli informacje o położeniu i kształcie obiektów na mapie.

Treść zadania wprost wymaga użycia danych przestrzennych, indeksów przestrzennych, procedur do ładowania i prezentowania danych oraz analiz przestrzennych i walidacji geometrii.

W tym projekcie:

- **park** jest zapisany jako **obszar**,
- **ścieżka** jest zapisana jako **linia**,
- **obiekt** w parku jest zapisany jako **punkt**.

To bardzo dobrze pasuje do rzeczywistości:

- park ma granice,
- ścieżka ma przebieg,
- ławka albo fontanna ma konkretną lokalizację.

Jakie pliki są w projekcie i za co odpowiadają

`tables_creation.sql`

To jest plik, który **tworzy bazę od strony struktury**.

Znajdują się w nim:

- włączenie rozszerzenia PostGIS,
- utworzenie tabel,
- utworzenie indeksów przestrzennych.

Można powiedzieć, że to jest „szkielet” całego projektu.

`proc.sql`

To jest najważniejszy plik z punktu widzenia logiki działania.

Są w nim zapisane procedury, czyli gotowe operacje, które można wykonywać na bazie.

Te procedury:

- dodają dane,
- sprawdzają poprawność danych,
- wyświetlają dane,
- analizują dane przestrzenne,
- budują geometrie z podanych punktów,
- przesuwają obiekty na mapie.

To jest „silnik” projektu.

`calls.sql`

To jest plik testowy.

Zawiera przykładowe wywołania procedur:

- dodawanie parków,
- dodawanie ścieżek,
- dodawanie obiektów,
- wyświetlanie danych,

- wyszukiwanie obiektów w promieniu,
- obliczanie długości i odległości,
- tworzenie geometrii z listy punktów,
- przesuwanie obiektu.

To jest plik, który pokazuje, że system naprawdę działa.

`Treść zadania 2.pdf`

To jest opis wymagań zadania.

Na jego podstawie można sprawdzić, czy projekt spełnia oczekiwania prowadzącego.

Jak działa baza danych krok po kroku

1. Włączenie obsługi danych przestrzennych

Na początku w pliku `tables_creation.sql` jest polecenie:

`CREATE EXTENSION IF NOT EXISTS postgis;`

To oznacza, że do zwykłego PostgreSQL dokładana jest obsługa map, geometrii i obliczeń przestrzennych.

Bez tego projekt nie mógłby przechowywać parków, ścieżek i punktów w postaci geometrii.

---

2. Utworzenie tabel

Projekt tworzy 3 główne tabele.

Tabela `parks`

Przechowuje informacje o parkach:

- `id` — identyfikator,
- `name` — nazwa parku,
- `area` — obszar parku jako wielokąt,

- `description` — opis.

Najważniejsze pole to `area`, bo ma typ:
`GEOMETRY(POLYGON, 4326)`

To znaczy:

- zapisujemy **wielokąt**,
- używamy układu współrzędnych **4326**.

W praktyce ten wielokąt opisuje granice parku.

Tabela `paths`

Przechowuje ścieżki:

- `id`,
- `park_id` — informacja, do którego parku należy ścieżka,
- `name`,
- `trail` — przebieg ścieżki jako linia.

Pole `trail` ma typ:
`GEOMETRY(LINESTRING, 4326)`

Czyli ścieżka jest linią złożoną z punktów połączonych odcinkami.

Tabela `facilities`

Przechowuje obiekty znajdujące się w parkach:

- `id`,
- `park_id`,
- `type` — rodzaj obiektu,
- `location` — położenie obiektu jako punkt.

Pole `location` ma typ:
`GEOMETRY(POINT, 4326)`

Czyli obiekt ma jedną konkretną pozycję na mapie.

---

3. Indeksy przestrzenne

Po utworzeniu tabel tworzone są indeksy GiST dla wszystkich pól geometrycznych:

- dla obszaru parku,
- dla przebiegu ścieżki,
- dla położenia obiektów.

Indeks przestrzenny działa podobnie do spisu treści w książce.

Dzięki niemu baza może szybciej znaleźć:

- które obiekty są blisko siebie,
- czy punkt znajduje się w parku,
- czy geometrie się przecinają.

Bez indeksów takie operacje byłyby wolniejsze.

---

4. Dodawanie parków

Za dodawanie parków odpowiada procedura `add_park(...)`.

Ta procedura:

1. przyjmuje nazwę, opis i kształt parku w formacie tekstowym,
2. zamienia ten tekst na geometrię,
3. sprawdza, czy geometria jest poprawna,
4. sprawdza, czy taki park już nie istnieje,
5. sprawdza, czy nowy park nie przecina się z istniejącym,
6. jeśli wszystko jest poprawne — zapisuje park do bazy.

To bardzo ważna procedura, bo pilnuje jakości danych.

Dzięki temu nie da się dodać parku:

- o błędnym kształcie,
- zdublowanego,

- nachodzącego na inny park.

Dla osoby mniej technicznej można to porównać do formularza z kontrolą poprawności:

- system sprawdza, czy wpis jest sensowny,
- nie pozwala dodać czegoś błędnego.

---

5. Dodawanie ścieżek

Za dodawanie ścieżek odpowiada procedura `add_path(...)`.

Ta procedura:

1. tworzy geometrię linii,
2. sprawdza, czy linia jest poprawna,
3. sprawdza, czy wskazany park istnieje,
4. sprawdza, czy cała ścieżka leży w granicach danego parku,
5. sprawdza, czy taka ścieżka już nie została dodana,
6. jeśli wszystko jest poprawne — zapisuje ścieżkę.

Czyli system pilnuje, aby ścieżka:

- należała do istniejącego parku,
- nie wychodziła poza park,
- nie była duplikatem.

To jest logiczne biznesowo — ścieżka przypisana do parku nie powinna leżeć poza jego granicami.

---

6. Dodawanie obiektów w parku

Za dodawanie obiektów odpowiada procedura `add_facility(...)`.

Ta procedura:

1. tworzy punkt na podstawie długości i szerokości geograficznej,
2. sprawdza poprawność punktu,

3. sprawdza, czy park istnieje,
4. sprawdza, czy punkt leży wewnątrz parku,
5. sprawdza, czy taki sam obiekt nie istnieje już w tej samej lokalizacji,
6. dodaje obiekt do bazy.

Dzięki temu nie można dodać np. fontanny do parku, jeśli jej pozycja leży poza jego granicami.

---

7. Wyświetlanie danych w przyjazny sposób

Treść zadania wymaga procedur do wyświetlania danych w sposób przyjazny użytkownikowi.

Projekt to realizuje przez kilka procedur.

`show_parks_with_info()`

Pokazuje listę parków razem z:

- nazwą,
- opisem,
- powierzchnią,
- punktem centralnym.

To jest wygodne, bo użytkownik nie musi sam liczyć powierzchni ani środka parku.

`show_paths_in_park(p_park_id)`

Pokazuje ścieżki w danym parku i ich długość.

W kodzie ta procedura została zapisana dwa razy, ale końcowo obowiązuje druga wersja, która pokazuje:

- ID ścieżki,
- nazwę,
- długość. filecite turn1file1 L128-L174

To nie psuje działania, ale warto wiedzieć, że wcześniejsza wersja została nadpisana.

`show_facilities_in_park(p_park_id)`

Pokazuje listę obiektów w danym parku wraz z ich współrzędnymi.

`show_facilities_nearby(...)`

Pokazuje obiekty znajdujące się w zadanym promieniu od wskazanego punktu.

`show_parks_containing_point(...)`

Sprawdza, do którego parku należy dany punkt.

Wszystkie te procedury są przyjazne użytkownikowi, bo zwracają gotowe informacje w zrozumiałej formie.

---

8. Analiza danych przestrzennych

To jest najciekawsza część projektu, bo pokazuje, że baza nie tylko przechowuje dane, ale potrafi też coś z nich wyliczyć.

Treść zadania wymaga analiz przestrzennych, takich jak powierzchnia, odległość czy lokalizacja obiektów w zakresie.

Projekt realizuje to w kilku miejscach.

Powierzchnia parku

W procedurze `show_parks_with_info()` użyta jest funkcja licząca powierzchnię parku.

Dzięki temu system może podać, jak duży jest park.

Długość ścieżek

Procedura `sum_paths_length_in_park(p_park_id)` liczy łączną długość wszystkich ścieżek w wybranym parku.

To pozwala odpowiedzieć np. na pytanie:

- ile metrów tras spacerowych znajduje się w danym parku?

Odległość między obiektami

Procedura `calculate_distance_between_facilities(...)` oblicza odległość między dwoma obiektami na podstawie ich położenia.

To pozwala np. sprawdzić:

- jak daleko jest od ławki do placu zabaw,
- jak daleko są od siebie dwa wybrane obiekty.

Szukanie obiektów w pobliżu

Procedura `show_facilities_nearby(...)` wyszukuje wszystkie obiekty znajdujące się w zadanym promieniu od wybranego punktu.

To można porównać do funkcji „pokaż, co jest w pobliżu”.

Sprawdzenie, w którym parku znajduje się punkt

Procedura `show_parks_containing_point(...)` sprawdza, czy wskazany punkt znajduje się wewnątrz któregoś parku.

To też jest klasyczna operacja przestrzenna.

---

9. Budowanie geometrii z punktów

Treść zadania dla oceny dobrej wymaga procedur do budowania geometrii oraz łączenia jej z układem współrzędnych.

Ten projekt spełnia to bardzo dobrze.

`create_park_from_points(...)`

Tworzy park z listy punktów granicznych.

Czyli zamiast ręcznie pisać cały wielokąt, można podać kolejne punkty granicy, a procedura sama zbuduje obszar parku.

`create_path_from_points(...)`

Tworzy ścieżkę z listy punktów.

`create_facility_from_point(...)`

Tworzy obiekt z jednego punktu zapisanego jako tekst.

To jest bardzo dobry element projektu, bo pokazuje, że użytkownik nie musi znać całej składni geometrii — może podać prostsze dane wejściowe.

---

10. Układ współrzędnych

W projekcie używany jest układ współrzędnych `4326`.

Widać to w definicjach tabel i procedurach tworzących geometrie.

Dla osoby mniej technicznej oznacza to tyle, że system zapisuje pozycje w standardzie zgodnym z mapami opartymi o długość i szerokość geograficzną.

Dzięki temu wiadomo, jak interpretować współrzędne.

W niektórych obliczeniach kod korzysta też z przekształcenia do układu `3857`, żeby liczyć metry, np. dla długości i odległości.

W praktyce chodzi o to, żeby system mógł zwracać wyniki typu:

- 300 metrów,
- 1385 metrów,
- 412 metrów.

---

11. Walidacja geometrii i bezpieczeństwo danych

Treść zadania dla oceny bardzo dobrej wymaga walidacji geometrii.

W tym projekcie walidacja jest naprawdę obecna i działa.

Przykłady:

- `add_park(...)` sprawdza, czy geometria parku jest poprawna i czy park nie przecina istniejącego parku.
- `add_path(...)` sprawdza, czy ścieżka jest poprawna i czy mieści się w granicach parku.
- `add_facility(...)` sprawdza, czy punkt jest poprawny i czy znajduje się w granicach parku.
- `translate_facility(...)` sprawdza, czy po przesunięciu obiekt nadal jest w parku i czy nie koliduje z innym obiektem.

To jest bardzo ważne, bo dzięki temu baza:

- nie przyjmuje błędnych danych,
- nie psuje spójności projektu,
- zachowuje sens przestrzenny.

---

12. Przesuwanie obiektu

Ciekawym dodatkiem jest procedura `translate_facility(...)`.

Ona:

1. znajduje wybrany obiekt,
2. przesuwa go o zadaną liczbę metrów,
3. sprawdza, czy po przesunięciu obiekt dalej leży w tym samym parku,
4. sprawdza, czy nie nachodzi na inny obiekt,
5. zapisuje nową pozycję.

To jest przykład bardziej zaawansowanej operacji przestrzennej.

Pokazuje, że system nie tylko zapisuje obiekty, ale potrafi też bezpiecznie zmieniać ich położenie.

---

13. Co pokazuje plik `calls.sql`

Plik `calls.sql` jest zbiorem przykładowych operacji uruchamianych na bazie.

Po kolei:

1. dodawane są 3 parki,
2. dodawane są 3 ścieżki,
3. dodawanych jest kilka obiektów w parkach,
4. wyświetlana jest lista parków,
5. wyświetlane są ścieżki w parkach,
6. wyświetlane są obiekty w parkach,

7. szukane są obiekty w pobliżu wybranego punktu,
8. sprawdzane jest, do którego parku należy punkt,
9. liczona jest łączna długość ścieżek,
10. liczona jest odległość między obiektami,
11. tworzony jest nowy park z listy punktów,
12. tworzona jest nowa ścieżka z punktów,
13. tworzony jest nowy obiekt z punktu,
14. przesuwany jest istniejący obiekt.

Ten plik jest ważny, bo pokazuje pełny zakres możliwości systemu.

Jak projekt spełnia wymagania zadania

Wymagania na ocenę dostateczną

1. Przechowywanie danych przestrzennych

Spełnione.

Tabele używają typów:

- `GEOMETRY(POLYGON, 4326)`,
- `GEOMETRY(LINESTRING, 4326)`,
- `GEOMETRY(POINT, 4326)`.

2. Wstępne uzupełnienie danych

Spełnione.

Plik `calls.sql` dodaje przykładowe parki, ścieżki i obiekty.

3. Indeksy przestrzenne

Spełnione.

Są utworzone indeksy GiST dla wszystkich geometrii.

4. Procedury do wyświetlania danych

Spełnione.

Są procedury pokazujące parki, ścieżki, obiekty i wyniki analiz.

5. Procedury do ładowania danych różnych typów geometrii

Spełnione.

Są procedury:

- `add_park` dla wielokątów,
- `add_path` dla linii,
- `add_facility` dla punktów.

Wymagania na ocenę dobrą

1. Procedury do budowania geometrii

Spełnione.

Są procedury:

- `create_park_from_points`,
- `create_path_from_points`,
- `create_facility_from_point`.

2. Powiązanie z geograficznym układem współrzędnych

Spełnione.

Projekt konsekwentnie używa SRID 4326 przy tworzeniu geometrii.

Wymagania na ocenę bardzo dobrą

1. Analiza danych przestrzennych

Spełnione.

Projekt liczy:

- powierzchnię parku,
- długość ścieżek,
- odległości między obiektami,
- obiekty w podanym promieniu,
- przynależność punktu do parku.

2. Walidacja geometrii

Spełnione.

Projekt sprawdza:

- poprawność geometrii,
- przecinanie parków,
- przynależność ścieżki do parku,
- przynależność obiektu do parku,
- poprawność przesunięcia obiektu.

Mocne strony projektu

Największe zalety tego rozwiązania są takie:

- temat jest logiczny i łatwy do zrozumienia,
- dobrze wykorzystano dane przestrzenne,
- są różne typy geometrii: punkt, linia, wielokąt,
- baza nie tylko przechowuje dane, ale też je analizuje,
- są zabezpieczenia przed błędnymi danymi,
- jest pokazane działanie na przykładach,
- projekt spełnia wymagania z treści zadania.

Drobna uwaga techniczna

W pliku `proc.sql` procedura `show_paths_in_park` została zdefiniowana dwa razy.

Druga definicja nadpisuje pierwszą.

To nie psuje działania projektu, ale warto o tym wspomnieć.

Po prostu końcowo używana jest druga, prostsza wersja procedury.

Podsumowanie

Ten projekt realizuje system zarządzania danymi przestrzennymi dla parków, ścieżek i obiektów parkowych.

Baza potrafi:

- przechowywać geometrie,
- pilnować poprawności danych,
- dodawać obiekty różnych typów,
- budować geometrie z punktów,
- analizować powierzchnię, długości i odległości,
- wyszukiwać obiekty w pobliżu,
- sprawdzać położenie względem parków.

Patrząc na wymagania z treści zadania, projekt spełnia:

- wymagania na ocenę dostateczną,
- wymagania na ocenę dobrą,
- oraz wymagania na ocenę bardzo dobrą. filecite turn1file3 L1-L25

Jest to rozwiązanie sensowne, kompletne i dobrze pokazujące, do czego służą dane przestrzenne w bazie danych.